

1. Set up an environment for integration testing and write test cases that validate the interaction between different components of a system?

1. What Is Integration Testing?

Integration testing verifies that different software components or modules work together as expected.

For example:

- **Frontend ↔ Backend**
 - **Backend ↔ Database**
 - **API ↔ Authentication service**
-

2. Set Up an Integration Testing Environment

Example Tech Stack:

- **Backend: Node.js / Django / Flask / Spring Boot**
- **Database: PostgreSQL / MySQL / MongoDB**
- **Frontend: React / Angular (optional for integration testing)**
- **Testing Frameworks:**
 - **Python: pytest, pytest-django, requests**
 - **JavaScript/Node.js: jest, supertest**
 - **Java: JUnit, Mockito, Spring Test**

Local Setup (Python Example):

```
bash
```

```
# Install dependencies
```

```
pip install pytest requests flask sqlalchemy
```

3. Sample Integration Test (Python + Flask Example)

Folder Structure:

```
bash
```

```
/myapp
```

```
├─ app.py
```

```
├─ database.py
```

```
├─ tests/
```

```
└─ test_integration.py
```

```
app.py
```

```
python
```

```
from flask import Flask, jsonify
```

```
from database import get_users
```

```
app = Flask(__name__)
```

```
@app.route("/users")
```

```
def users():
```

```
    return jsonify(get_users())
```

```
database.py
```

```
python
```

```
def get_users():  
    return [{"id": 1, "name": "Alice"}, {"id": 2, "name": "Bob"}]
```

tests/test_integration.py

python

```
import pytest
```

```
import app
```

```
@pytest.fixture
```

```
def client():
```

```
    app.app.config["TESTING"] = True
```

```
    with app.app.test_client() as client:
```

```
        yield client
```

```
def test_users_endpoint(client):
```

```
    response = client.get("/users")
```

```
    assert response.status_code == 200
```

```
    data = response.get_json()
```

```
    assert isinstance(data, list)
```

```
    assert data[0]["name"] == "Alice"
```

4. Sample Test Cases to Validate Integration

Component A	Component B	What to Test
API Route	Database	Data fetch from DB and correct format
Auth Service	API Gateway	Token validation and error handling
Microservice A	Microservice B	Data handoff, HTTP status, schema
Frontend	REST API	JSON data binding, error rendering

5. Run the Tests

```
bash
pytest tests/
```

6. Expected Output

```
bash
===== test session starts =====
collected 1 item

tests/test_integration.py .          [100%]

===== 1 passed in 0.12s =====
```